

Production Operations

Environment separation

Use the same NestJS application architecture with separate values and secret stores:

Environment	Database	Storage	Required operational checks
local	sqlite / local.db	local-test	migrations, auth, course, lesson, admin flows
staging	d1	r2	Cloudflare bindings, auth, upload, private media, integration
production	d1	r2	HTTPS/domain, secrets, analytics, monitoring, rollback

Never copy production values into `.env`, tests, screenshots, logs, documentation, or a client bundle. `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`, `CLOUDFLARE_API_TOKEN`, `JWT_SECRET`, and admin bootstrap password are server-only.

Controlled admin bootstrap

There is no public admin registration route. Run this command only from a controlled operator shell with the target environment's secret store attached:

```
ADMIN_BOOTSTRAP_EMAIL='admin@example.invalid' \  
ADMIN_BOOTSTRAP_PASSWORD='set-in-secret-store-not-shell-history' \  
npm run seed:admin
```

The command runs pending migrations, hashes the password through `PasswordService`, inserts `role='admin'`, and prints only `created` or `already_exists`. It is idempotent for an existing admin. It refuses to change an existing non-admin account. Do not place either variable in a committed file or command history.

Rollback procedure

1. Stop promotion and record the deployment identifier, migration filename, and incident timestamp. Do not print or copy secret values.
2. Revert the application deployment to the last known-good immutable build using the hosting provider's deployment rollback command. T19 does not claim a live provider deployment; execute this only in the configured staging/production platform.
3. Confirm the reverted build starts with the target environment's secret store and run the health check. Check logs for event names and status only.
4. If the failed release applied the latest migration, restore a database backup/snapshot taken before that migration. The repository migration runner is forward-only; it does not invent down migrations.
5. If the platform requires an explicit compensating migration, create a new numbered migration that reverses the affected schema change. Never edit an applied migration because `_migrations` hash drift must fail closed.
6. Re-run the migration runner against a disposable local copy first, then verify `PRAGMA foreign_key_check` and the affected application flow.
7. Re-deploy the last known-good build, verify auth, admin authorization, storage access, and error responses, and record evidence without credentials.
8. Keep the failed deployment isolated until the root cause and a forward fix are reviewed. Do not claim production recovery from a local dry run.

Observability boundary

Application errors use the local `MonitoringPort` interface in `src/observability/monitoring.port.ts`; no provider SDK or credential is selected in T19. The provider decision remains open. Logs and monitoring events must contain event names, route/status context, and non-sensitive identifiers only. They must not contain passwords, tokens, hashes, signed URLs, R2 credentials, or exception messages copied from untrusted input.

